AI-SOC: Work Division Plan for Two People

Major project · B.Tech CSE, University of Mumbai · Plan date 28 Oct 2026 · Companion to the AI-SOC Major Project Plan (dates and exit gates unchanged)

1. How the work is split

The split follows **ownership boundaries**, not phases. **Person A** owns everything that *detects and acts* (the safety-critical backend). **Person B** owns everything that *thinks and shows* (the LLM, evaluation, login, approvals and the user interface). Every pull request is reviewed by the other person, so both can explain the whole system in the viva. The labels A and B are interchangeable: assign them by who is stronger in backend or AI work.

PERSON A – Platform, Policy and Response	PERSON B – Intelligence, Security and Experience
• **Packages:** ingestion, detection, incident, policy, remediation, audit • Windows monitor (reliability, API key, retry) • Correlation, rules R1 to R3, incident lifecycle • Policy engine, response plan, action state machine • Hash-chained audit log • Remediation executor, lab VMs, firewall block with TTL, verification, rollback, kill switch • CI pipeline, Flyway migrations, environment secrets	• **Packages:** investigation, approval, security; whole frontend; evaluation harness • Ollama service, evidence pack, prompt, JSON schema • Validators (schema, grounding, consistency) and fallback • Spring Security, roles, approval workflow and inbox • Timeline, ATT&CK lookup, prompt-injection tests • Scenario dataset, ablation runs, model comparison • Analytics dashboard, full Docker Compose

2. Phase-by-phase assignment

Dates run from 2 Nov 2026 to 25 Apr 2027. Adjust to your college calendar.

Phase / dates	Person A	Person B	Joint work and gate
Phase 0 **2–8 Nov**	• CI pipeline, Flyway baseline, environment secrets, .env.example • Lab VM that produces a 4625 with a real source IP	• Audit frontend and investigation code • Frontend build added to CI	Both: agree interface contracts and OpenAPI; merge audits into one README status table. Gate: CI green, source IP confirmed
Phase 1 **9–22 Nov**	• Monitor bookmark, retry, API key • Idempotent ingest, correlation, rules R1 to R3 • Incident lifecycle and edge-case tests	• Incident fixtures so work does not wait on A • Scenario dataset v1 • Dashboard cleanup; prototype prompt and schema against Ollama	Gate: 200 replayed alerts give correct incidents, zero duplicates
Phase 2 **23 Nov–13 Dec**	• Start policy engine against the agreed investigation schema • Docker Compose for Postgres and backend	• OllamaInvestigationService, evidence pack, prompt v1, JSON schema • Validators, fallback, stubbed-Ollama tests, UI source badge • Evaluation harness v1 with 30 scenarios	Gate: every failure mode ends in a safe fallback; first accuracy table
Stage I **14–20 Dec**	• Report: ingestion, detection, design chapters	• Report: literature survey, investigation design	Both: presentation and demo. Milestone: working pipeline with real LLM investigation
Buffer **21 Dec–3 Jan**	• Tests and docs only	• Tests and docs only	No new features
Phase 3 **4–17 Jan**	• Finish policy engine and response plan • Action state machine • Hash-chained audit log	• Start Spring Security and user model early • Write audit events from investigation	Gate: block of a private or allow-listed IP is always DENY; every decision audited
Phase 4 **18–31 Jan**	• Executor skeleton with a test-only approved action • Review authorization tests	• Roles, API key validation, approval workflow • Approval inbox and audit view • Authorization test matrix	Gate: unapproved action never reaches executor; forged alert rejected

Phase / dates	Person A	Person B	Joint work and gate
Phase 5 1–21 Feb	• Isolated lab, executor, firewall block with TTL • Verification, rollback, kill switch • Abuse and failure tests	• Remediation status UI, approval-to-execution wiring • End-to-end browser tests • Seeded replay script for the demo	Gate: scripted attack leads to an approved, verified block, removed on expiry
Phase 6 22 Feb–7 Mar	• Extra evidence fields for B • Confirm model text cannot change allowed actions	• Evidence provenance, incident timeline • ATT&CK lookup from local file • Prompt-injection test set and fixes	Gate: injection scenarios cause no policy bypass
Phase 7 8–21 Mar	• Scenarios and metrics for detection, policy, remediation	• Dataset to 80–100 scenarios, ablation runs, model comparison • Analytics dashboard, full Docker Compose	FEATURE FREEZE 21 Mar
Phase 8 22 Mar–25 Apr	• Report chapters, demo part 1, viva preparation	• Report chapters, demo part 2, viva preparation	Both: final report, slides, demo video, tag v1.0, two weeks of buffer

3. Interfaces to agree in week 1 (by 8 Nov)

Write these as shared schemas plus an OpenAPI file before either person builds on them. After 8 Nov, changing any of them needs both people to approve the pull request.

#	Interface	Owner	Counterpart
1	**Alert schema**	A defines	B consumes (fixtures, evidence pack)
2	**Incident evidence object (incident, alerts, computed facts)**	A provides	B consumes
3	**Investigation result JSON schema**	B defines	A's policy engine consumes
4	**Policy decision and response plan format**	A defines	B shows in the UI
5	**Action state machine and approval API**	A owns state enum and executor	B owns approval endpoints and UI
6	**Audit event format**	A defines	B writes events from investigation, approval, login

4. Unblocking rules

- B builds against fixture incidents until A's Detection v2 lands. A uses the mock investigation, rebuilt from real incident data, until B's Ollama service lands.
- A tests the executor with a test-only approved action until B's approval workflow lands.
- Anything not ready by its gate is stubbed behind the agreed interface, never worked around.
- B is lighter in Phase 1 and A is lighter in Phase 2, so each starts the other's upcoming work early (fixtures and dataset for B, policy engine for A).

5. Working agreement

- Short-lived feature branches; every pull request reviewed by the other person; CI must be green before merge; CODEOWNERS file by package.
- Flyway migrations use date-based version numbers to avoid clashes. Shared hotspot files (application.properties, App.tsx, the AIInvestigation entity) change only in small, quick pull requests.
- Two short syncs per week plus a one-hour integration session every Friday: run main end to end, check the gate, and move tasks between people if either is more than one week behind.
- Keep a contribution log of who built what, for any individual contribution statement your college requires.
- After each phase, each person teaches the other their part for 15 minutes.
- Push daily; tag every milestone (v0.1 baseline, v0.5 Stage I, v1.0 final).

6. Report, demo and viva split

	Person A	Person B
Report chapters	Ingestion and detection design; policy and remediation design; lab and safety; results for detection and remediation	Literature survey on LLMs in security operations; investigation design; authentication and approvals; evaluation method and investigation results
Both write	Introduction, architecture, limitations, conclusion. Each person reads the full report before submission.	
Demo	Runs the attack and remediation half: failed logons, one incident, approved block, verification, rollback	Runs the investigation and approval half: LLM result with cited evidence, policy decision, approval, audit trail, negative cases
Viva	Both prepare every likely question: why a local model, why the LLM does not decide, prompt injection, what happens when Ollama is down, how accuracy was measured, what stops a forged alert from causing a block.	

7. What the second person buys

- Keep the dates. Use the extra capacity to promote "Should" items into the core scope, in this order: ATT&CK lookup, model comparison, rules R2 and R3, analytics dashboard.
- Re-check the scope cut line at the Stage I milestone in mid-December.
- Safety rules are unchanged: remediation only on isolated lab VMs, default DRY_RUN, protected allow-list, TTL on every block, global kill switch, no secrets in Git.